

Android App Additional Components

Additional Components of Android Application

Some Additional Components of Android Application

1. Fragments
2. Intents
3. Widgets
4. Views
5. Notifications
6. Layout XML Files
7. App APK Files
8. Resources

1. Fragment: sub-activity

- A **Fragment** is a piece of an activity which enables more modular activity design. A fragment is a kind of **sub-activity**.
- We create fragments by extending **Fragment** class and we can insert a fragment into our activity layout by declaring the fragment in the activity's layout file, as a **<fragment>** element.

Fragment: sub-activity...

Background

- Prior to fragment, we have been facing limitation because we could show only one activity on the screen at one given point in time. So we were not able to divide device screen and control different parts separately.
- But with the introduction of fragment we got more flexibility and removed the limitation of having a single activity on the screen at a time.
- Now we can have a single activity but each activity can comprise of multiple fragments which will have their own layout, events and complete life cycle.

Fragment: sub-activity...

Following are important points about fragment –

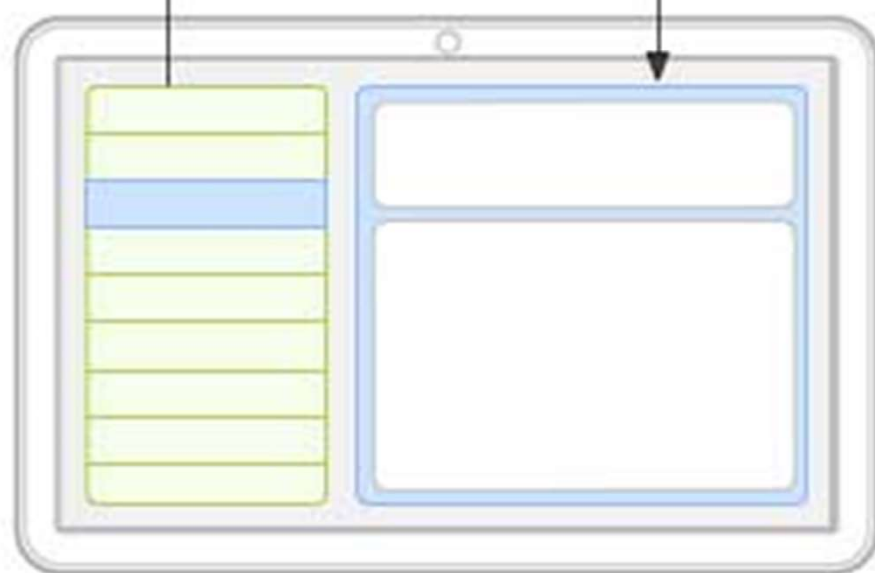
- A fragment has its own layout and its own behaviour with its own life cycle callbacks.
- You can add or remove fragments in an activity while the activity is running.
- You can combine multiple fragments in a single activity to build a multi-pane UI.
- A fragment can be used in multiple activities.

Fragment: sub-activity...

- Fragment life cycle is closely related to the life cycle of its host activity which means when the activity is paused, all the fragments available in the activity will also be stopped.
- A fragment can implement a behaviour that has no user interface component.
- Fragments were added to the Android API in Honeycomb version of Android which API level 11.

Tablet

Selecting an item
updates Fragment B



Activity A contains
Fragment A and Fragment B

Handset

Selecting an item
starts Activity B



Activity A contains
Fragment A



Activity B contains
Fragment B

Fragment: sub-activity...

- The application can embed two fragments in Activity A, when running on a tablet-sized device.
- However, on a handset-sized screen, there's not enough room for both fragments, so Activity A includes only the fragment for the list of articles, and when the user selects an article, it starts Activity B, which includes the second fragment to read the article.

Fragment: Callback Methods

- **onAttach()** The fragment instance is associated with an activity instance. The fragment and the activity is not fully initialized. Typically we get in this method a reference to the activity which uses the fragment for further initialization work.
- **onCreate()** The system calls this method when creating the fragment. We should initialize essential components of the fragment that we want to retain when the fragment is paused or stopped, then resumed.

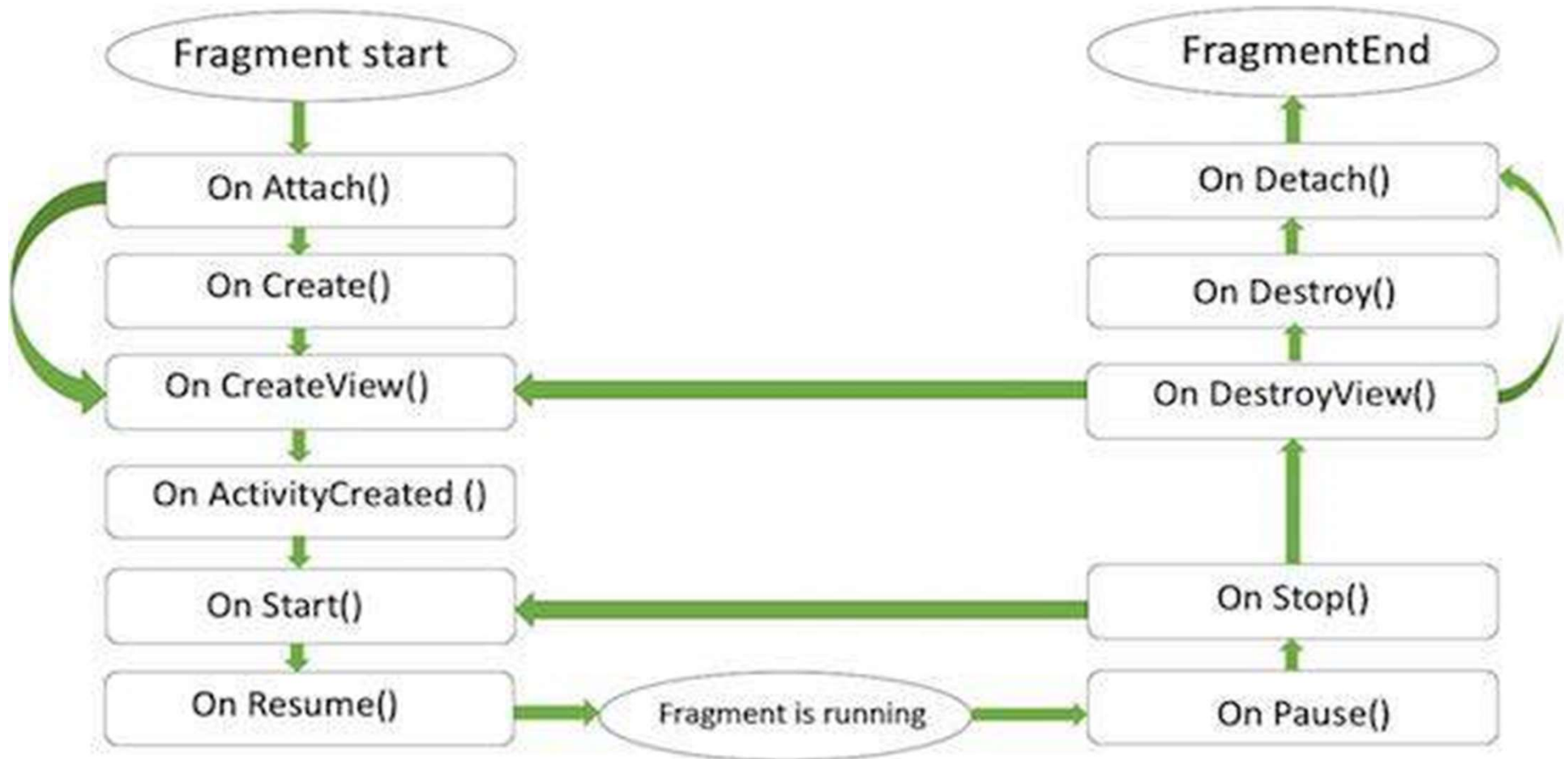
Fragment: Callback Methods

- **onCreateView()** The system calls this callback when it's time for the fragment to draw its user interface for the first time.
- **onActivityCreated()** This is called after the onCreateView() method when the host activity is created. Activity and fragment instance have been created as well as the view hierarchy of the activity.
- **onStart()** The onStart() method is called once the fragment gets visible.
- **onResume()** Fragment becomes active.

Fragment: Callback Methods

- **onPause()** The system calls this method as the first indication that the user is leaving the fragment.
- **onStop()** Fragment going to be stopped by calling onStop().
- **onDestroyView()** Fragment view will destroy after call this method.
- **onDestroy()** This is called to do final clean up of the fragment's state but Not guaranteed to be called by the Android platform.

Fragment Life Cycle



How to Use Fragments

1. First of all decide how many fragments we want to use in an activity.
2. Next based on number of fragments, create classes which will extend the *Fragment* class. The *Fragment* class has above mentioned callback functions. We can override any of the functions based on our requirements.
3. Corresponding to each fragment, we will need to create layout files in XML. These files will have layout for the defined fragments.
4. Finally modify activity file to define the actual logic of replacing fragments based on our requirement.

Types of Fragments

Basically fragments are divided as three stages.

1. **Single frame fragments** - Single frame fragments are used in hand hold devices like mobiles, here we can show only one fragment as a view.
2. **List fragments** - fragments having special list view is called as list fragment
3. **Fragments transaction** - Using with fragment transaction we can move one fragment to another fragment.

2. Intents

- The **dictionary meaning** of intent is *intention or purpose*. So, it can be described as the intention to do an action.
- **Android Intent** is the *message* that is passed between components such as activities, services, content providers, broadcast receivers, etc.
- It can be used with **startActivity** to launch an Activity, **broadcastIntent** to send it to any interested BroadcastReceiver components, and **startService(Intent)** or **bindService(Intent, ServiceConnection, int)** to communicate with a b/g Service.

Intents...

Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Intents...

Types of Intents: There are two types of intents supported by Android:

1. Implicit and 2. Explicit.

1. Implicit Intent

These intents do not name a target and the field for the component name is left blank. Implicit intents are often used to activate components in other applications.

```
Intent read1=new Intent();  
read1.setAction(android.content.Intent.ACTION_VIEW);  
read1.setData(ContactsContract.Contacts.CONTENT_URI);  
startActivity(read1);
```

Intents...

Implicit Intent

It specifies the only action to be performed and does not directly specify Android Components.

Here we just mention the action in the intent and OS decides which applications are suitable to handle the task, action across two different applications.

Example: When we tap the share button in any app we can see the Gmail, Bluetooth, and other sharing app options. Here user sends a request is the implicit intent request which can be handle by these Gmail, Bluetooth-like app.

Intents...

2. Explicit Intents

- Explicit intents are used for communication inside the application. Like changing activities inside the application.
- Explicit intent explicitly specifies the component. In such case, intent provides the external class to be invoked.

```
Intent i = new Intent(getApplicationContext(), ActivityTwo.class);  
startActivity(i);
```

3. Widgets

Widgets are variations of Broadcast Receivers and essential aspects of home screen customization. They display data and allow users to perform actions on them. There are various types of widgets:

1. Information widget
2. Collection widget
3. Control widget
4. Hybrid widget



Widgets

- Information widget: These widgets display crucial information and track how the information changes over time. Example - Clock widgets and widgets that display weather and time information.
- Collection widget: As the name depicts, collection widgets are a collection of information of the same type. Its use is for browsing information and opening any one of the elements to view details. Example - music widgets, as we can skip pause and play music outside the music application.

Widgets

- Control widget: These widgets display functionalities and by using them, the user can trigger from home screen without opening the application. Example - pause and play the video outside the application.
- Hybrid widget: These widgets combine features of all the other three widgets. Example - music player widget is a control widget but it also informs the user about which track is playing currently, which means it is a combination of control and information thus it is termed as hybrid widget.

4. Views

Views are responsible for drawing and event handling. They are rectangular elements on the screen. Some of the views are EditText, ImageView Button, CheckBox and ImageButton.

5. Notifications

Notifications alert users when the application is not visible or is inactive. This alert flashes on the screen and then disappears.

Example - Notification of the new incoming message popped on the screen.

6. Layout XML Files

Layout is the structure for the user interface in the application. XML files provide different types of layouts for the different type of screen, it also specifies which GUI component, an activity or fragment holds.

7. App APK Files

Apk file is the package file format that contains the program's code, resources, assets. The Android operating system uses them for installing mobile applications and middleware.

8. Resources

Resources are the additional files and static content that your code uses, such as bitmaps, layout definitions, user interface strings, animation instructions, and more.

Resources in Android is for defining Images, texts, string values. Everything is defined in the resource file and it can be referenced within the source code.

We should always externalize app resources such as images and strings from our code, so that we can maintain them independently.

Thank You